DNN - Project 1 - Report

Jakub Pniewski 459 481

November 2025

1 Introduction

The purpose of the task was to train a model with a common CNN backbone and 2 heads:

- head for classification into one of 135 classes
- head for regression on 6 classes

2 Augmentation

In order to increase the amount of data, I implemented 3 augmentors:

- random horizontal flip
- random vertical flip
- random 90 degree rotation

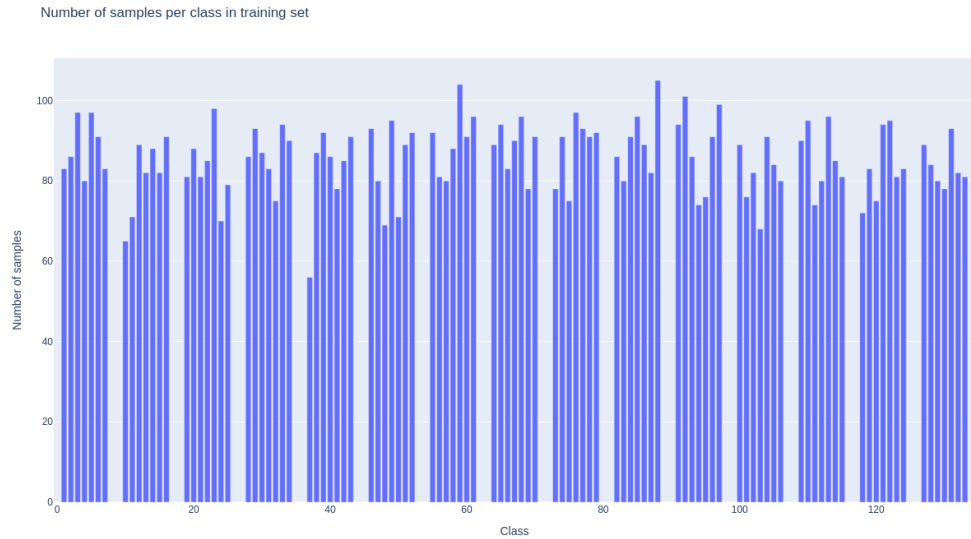
I set the probabilities of applying the first two to 0.5 and the third one to 2/3. The reason for applying the rotation with this probability is that I randomly choose whether to rotate clockwise or counterclockwise. This way, there is a uniform probability distribution over three cases: clockwise rotation, counterclockwise rotation, and no rotation.

The augmentations are being composed; for example, with a probability of 1/4, we have both horizontal and vertical flips. All of these augmentations affect the labels. For example, in a horizontal flip, right-oriented triangles are swapped with left-oriented ones. In fact, any orientation of a triangle has an equal chance of becoming any other.

These three augmentations seemed the most reasonable. I am sure that I'm feeding good labels to the model. Changing brightness or contrast would introduce noise, as all the real images have binary pixels. Gaussian noise and random erasing would affect the images in a way that a different label might be appropriate. For example, we may erase the tip of a triangle, so the label corresponding to the number of triangles should be decreased. However, we wouldn't be able to tell that, so we would be teaching our model an incorrect label.

3 Exploratory Data Analysis

I decided to check the number of occurrences of each class in the dataset. The distribution I found is fairly close to uniform and can be seen in the image below.



This distribution will become even smoother when we apply the augmentations. This is because, in the augmentations, every directed triangle can become any other with equal probability. The only aspect of that bar plot that we might be concerned about is the "gaps" between bars. This means that some labels are not present at all in our dataset.

The other thing I checked was the distribution in the regression task. To do that, I counted how many times each number from 1 to 9 appears in a pair in the labels of the dataset. The results can be seen below.

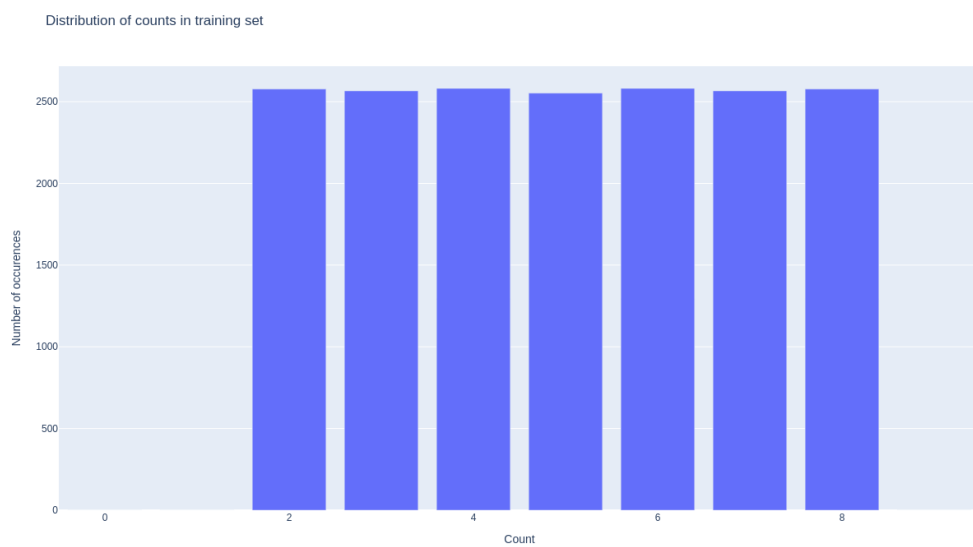


Figure 1: Occurences of counts of shapes

From this plot, we can clearly see that in our dataset there are no pairs containing 1 and 9 (for example, we never have one square and nine circles). This also explains the gaps in the previous distribution, which perfectly align with this observation when we inspect the way I map the original labels to classes.

I also checked the second distribution separately for each of the six regression classes, and the results were the same. This means we don't have to scale the classes or the loss in any way to achieve good results.

4 Model architecture

I use the CNN backbone required by the problem statement:

```
backbone = nn.Sequential(
    nn.Conv2d(1, 8, 3, stride=1, padding=1), nn.ReLU(),
    nn.Conv2d(8, 16, 3, stride=1, padding=1), nn.ReLU(),
    nn.Conv2d(16, 32, 3, stride=1, padding=1), nn.ReLU(),
    nn.Conv2d(32, 64, 3, stride=1, padding=1), nn.ReLU(),
    nn.Flatten(start_dim=1),
    nn.Linear(64 * 28 * 28, 256), nn.ReLU()
)
```

Before feeding an image to the backbone, I scale it so that white and black pixels are -1 and 1 instead of 0 and 255. I define two similar heads:

```
head_cls = nn.Sequential(
    GeGLU(256, dropout),
    GeGLU(256, dropout),
    GeGLU(256, dropout),
    nn.Dropout(dropout),
    nn.Linear(256, n_cls_classes),
    nn.LogSoftmax(dim=1)
)
self.head_cnt = nn.Sequential(
    GeGLU(256, dropout),
    GeGLU(256, dropout),
    GeGLU(256, dropout),
    nn.Dropout(dropout),
    nn.Linear(256, n_cnt_classes)
)
```

GeGLU is a well-known module, widely used in NLP. More about it can be found in the ArXiv paper. I added LayerNorm and dropout to improve regularization.

5 Training

I conducted three trainings described in the problem statement. I had to choose two very important variables - weight decay and dropout rate. They were crucial as we are working on a very small dataset and really want to avoid overfitting.

I also checked a few values of λ_{cnt} and ended up using $\lambda_{cnt} = 1$. For values in the range $[0.5, 5]$, the results were similar. For larger $\lambda_{cnt} > 5$, the classification head performed worse, and for $\lambda_{cnt} < 0.5$, the regression learned too slowly. Within the range I provided, the results were more or less the same.

5.1 Multitask training

Metric	**Value**
Runtime	442.18 seconds
Final regression eval loss	0.1200
Final classification eval loss	1.2040
Final accuracy	0.5480
Final RMSE	0.52

5.2 Classification only

Metric	Value
Runtime	438.91 seconds
Final classification eval loss	1.2219
Final accuracy	0.499

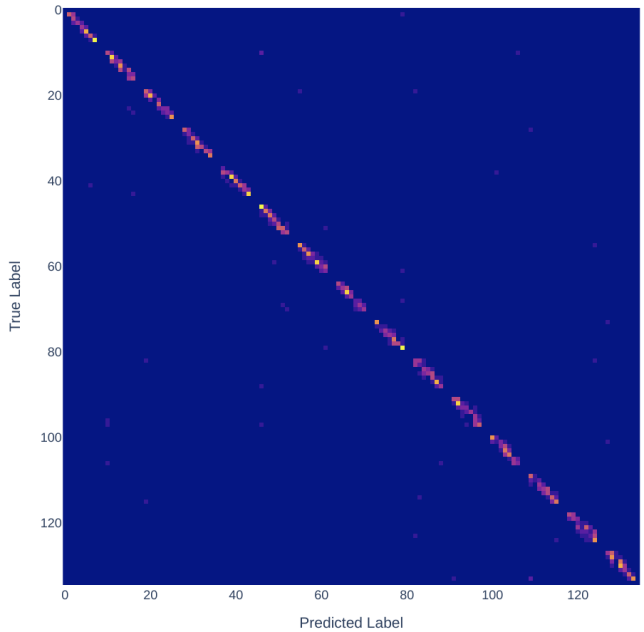
5.3 Regression only

Metric	Value
Runtime	443.73 seconds
Final regression eval loss	0.1716
Final RMSE	0.64

6 Results

In my case, the results from multitask learning are better than the results from the other two experiments. I suppose we benefit from two aspects. The first is transfer learning. The second is a form of regularization - it's harder for the model to overfit to one task when there are other updates occurring that do not apply to that task. The results described above may be slightly different from the Colab version, as I didn't apply early stopping in order to obtain full training statistics.

To show my results, I decided to additionally plot a confusion matrix. I used the multitask-trained model, as it turned out to be the best one.



We can see that there are many misclassifications that are very close to the diagonal. I decided to inspect the examples with errors and found that for 394 out of 444 misclassified samples, my model actually predicted a correct pair, but with an incorrect split of elements.